

Conflict Serializability — Exam Notes

Precedence Graphs, Edge Construction, Cycle Testing & Topological Sorting

1. What is Conflict Serializability?

A non-serial schedule is **conflict serializable** if it is conflict equivalent to some serial schedule. This means we can swap adjacent non-conflicting operations until all operations of each transaction are grouped together consecutively.

★ Golden Rule for Exams

Precedence Graph is **ACYCLIC (No Cycle)** → **Conflict Serializable** ✓

Precedence Graph has **CYCLE** → **NOT Conflict Serializable** ✗

2. Standard 6-Step Solving Method

- 1. Identify Transactions:** Count total transactions ($T_1, T_2, \dots, T_n$).
- 2. Create Vertices:** Place one vertex (node) for each transaction in the precedence graph.
- 3. Identify Conflict Pairs:** Find operations from *different transactions* operating on the *same data item* where at least one is a **WRITE**.
- 4. Draw Directed Edges:** If conflicting operation in T_i occurs **BEFORE** T_j , draw directed edge: $T_i \rightarrow T_j$.
- 5. Cycle Detection:** Check if there is any closed loop path in the graph.
- 6. Determine Result & Order:** If acyclic, use Topological Sort (Indegree = 0) to find the serial order.

3. Conflict Pair Matrix & Trigger Conditions

First Operation (T_i)	Second Operation (T_j)	Same Data?	Conflict Status	Edge Generated
$R_1(A)$	$R_2(A)$	Yes	✗ Non-Conflict	None
$R_1(A)$	$W_2(A)$	Yes	● Conflict (RW)	$T_1 \rightarrow T_2$
$W_1(A)$	$R_2(A)$	Yes	● Conflict (WR)	$T_1 \rightarrow T_2$
$W_1(A)$	$W_2(A)$	Yes	● Conflict (WW)	$T_1 \rightarrow T_2$
$R_1(A)$	$W_2(B)$	No ($A \neq B$)	✗ Non-Conflict	None

4. Precedence Graph Construction Rules

Rule A: Vertices Formula

Number of Graph Vertices = Total Transactions

$$|V| = \text{Number of Transactions}$$

Rule B: Edge Direction

From **earlier** transaction operation to **later** conflicting operation.

$$T_{\text{earlier}} \rightarrow T_{\text{later}}$$

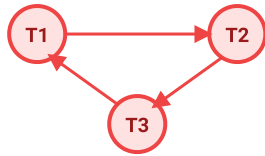
⚡ Special Checking Trick for WRITE Operations

When evaluating a **READ** operation $R(X)$, check subsequent operations for: $W(X)$.

When evaluating a **WRITE** operation $W(X)$, check subsequent operations for BOTH: $R(X)$ and $W(X)$.

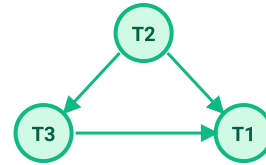
5. Graph Comparison: Cycle vs. Acyclic

● Cyclic Graph (NOT Serializable)



Cycle: $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$ (Failed ❌)

✓ Acyclic Graph (Serializable)



No Cycle: Path $T2 \rightarrow T3 \rightarrow T1$ (Passed ✓)

6. Finding Equivalent Serial Order (Topological Sort)

If the precedence graph is acyclic, use the **Indegree Method** to find the equivalent serial order:

1. Calculate **Indegree** (number of incoming edges) for every vertex.
2. Select the vertex with **Indegree = 0** (e.g., T_2). This executes first.
3. Remove that vertex and all its outgoing edges from the graph.
4. Recalculate indegrees for remaining vertices and repeat until all nodes are placed.

Step	Current Graph State	Node with Indegree = 0	Action / Added to Order
1	Full Graph ($T_2 \rightarrow T_3$, $T_2 \rightarrow T_1$, $T_3 \rightarrow T_1$)	T_2 (Indegree = 0)	Select T_2 , remove outgoing edges
2	Remaining Graph ($T_3 \rightarrow T_1$)	T_3 (Indegree = 0)	Select T_3 , remove outgoing edges
3	Remaining Graph (T_1)	T_1 (Indegree = 0)	Select T_1

Resulting Equivalent Serial Order: **$T2 \rightarrow T3 \rightarrow T1$**

7. Total Serial Arrangements Formula

For n transactions, the total possible serial arrangements is given by:

Total Serial Schedules = $n!$ For $n = 3 \rightarrow 3! = 6$ possible serial orders

8. 🌟 High-Yield Exam Traps & GATE Quick Summary

Critical Exam Traps to Avoid:

- **Same Transaction Trap:** Operations belonging to the SAME transaction (, $R_1(X)$ followed by $W_1(X)$) NEVER create a self-loop edge ($T_1 \rightarrow T_1$). Ignore intra-transaction pairs!
- **Different Variable Trap:** $R_1(X)$ and $W_2(Y)$ do NOT conflict even if one is a write, because $X \neq Y$.

🏆 Final Exam Revision Formula ($C \rightarrow E \rightarrow C \rightarrow S$)

Conflicts (RW, WR, WW on same item across diff transactions)

→ **Edges** (From earlier transaction to later transaction)

→ **Cycle Check** (Acyclic = Serializable | Cyclic = Not Serializable)

→ **Serial Order** (Topological sort using Indegree = 0)